Traducido del inglés al español - www.onlinedoctranslator.com

Este activo ha sido archivado.

Arquitectura para la nube

Prácticas recomendadas de AWS

octubre 2018

Para obtener información sobre prácticas recomendadas, consulte el

Marco de buena arquitectura de AWS k.

Archivado

Avisos

Este documento es provisto únicamente con fines informativos. Representa las ofertas y prácticas actuales de productos de AWS a la fecha de emisión de este documento, que están sujetas a cambios sin previo aviso. Los clientes son responsables de realizar su propia evaluación independiente de la información contenida en este documento y cualquier uso de los productos o servicios de AWS, cada uno de los cuales se proporciona "tal cual" sin garantía de ningún tipo, ya sea expresa o implícita. Este documento no crea garantías, representaciones, compromisos contractuales, condiciones o garantías de AWS, sus afiliados, proveedores o licenciantes. Las responsabilidades y obligaciones de AWS con sus clientes están controladas por los acuerdos de AWS, y este documento no forma parte ni modifica ningún acuerdo entre AWS y sus clientes.

© 2018, Amazon Web Services, Inc. o sus afiliados. Reservados todos los derechos.

Archivado

Contenido

Introducción 1

Diferencias entre los entornos de computación tradicional y en la nube 2

- Activos de TI como recursos aprovisionados 2
- Capacidad global, disponible y escalable 2
- Servicios gestionados de nivel superior 3
- Seguridad incorporada 3
- Arquitectura por costo 3
- Operaciones en AWS

Criterios de diseño 5

- Escalabilidad 5
- Recursos desechables en lugar de servidores fijos 9
- Automatización 12
- Bajo acoplamiento 14
- Servicios, no servidores 18
- bases de datos 20
- Gestión de volúmenes crecientes de datos 27
- Eliminación de puntos únicos de falla 27
- Optimizar por costo 33
- almacenamiento en caché 36
- Seguridad 37

Conclusión 41

Colaboradores 41

Archivado

Otras lecturas 41

Revisiones de documentos 42

Archivado

Abstracto

Este documento técnico está destinado a arquitectos y desarrolladores de soluciones que están creando soluciones que se implementarán en Amazon Web Services (AWS). Brinda orientación arquitectónica y asesoramiento sobre patrones de diseño técnico y cómo se aplican en el contexto de la computación en la nube. Esta introducción proporciona los conceptos clave y las diferencias al diseñar soluciones en AWS. Incluye una discusión sobre cómo aprovechar los atributos que son específicos de la naturaleza dinámica de la computación en la nube, como la elasticidad y la automatización de la infraestructura. Estos patrones pueden proporcionar el contexto para una revisión más detallada de las opciones, el estado operativo y el estado de implementación como se detalla en el *Marco de buena arquitectura de AWS*.[1]

Archivado

Introducción

Migrar aplicaciones a AWS, incluso sin cambios significativos (un enfoque conocido como *levantar y cambiar*), brinda a las organizaciones los beneficios de una infraestructura segura y rentable. Sin embargo, para aprovechar al máximo la elasticidad y la agilidad que son posibles con la computación en la nube, los ingenieros deben desarrollar sus arquitecturas para aprovechar las capacidades de AWS.

Para las nuevas aplicaciones, los patrones de arquitectura de TI específicos de la nube pueden ayudar a impulsar la eficiencia y la escalabilidad. Esas nuevas arquitecturas pueden admitir cualquier cosa, desde análisis en tiempo real de datos a escala de Internet hasta aplicaciones con tráfico impredecible de miles de Internet de las cosas (IoT) o dispositivos móviles conectados.

Ya sea que esté rediseñando las aplicaciones que actualmente se ejecutan en su entorno local para que se ejecuten en AWS, o diseñando aplicaciones nativas de la nube, debe comprender las diferencias entre los entornos tradicionales y los entornos de computación en la nube. Esto incluye opciones de arquitectura, escalabilidad, tipos de recursos, automatización, así como componentes, servicios y bases de datos flexibles. Si es nuevo en AWS, le recomendamos que revise la información en el Acerca de AWS página para obtener una comprensión básica de los servicios de AWS.[2]

Archivado

Diferencias entre los entornos de computación tradicional y en la nube

La computación en la nube se diferencia de un entorno local tradicional en muchos aspectos, incluida la capacidad flexible, global y escalable, los servicios administrados, la seguridad integrada, las opciones para la optimización de costos y varios modelos operativos.

Activos de TI como recursos aprovisionados

En un entorno informático tradicional, aprovisiona la capacidad en función de una estimación de un pico máximo teórico. Esto puede resultar en períodos en los que los recursos costosos están inactivos o en ocasiones de capacidad insuficiente. Con la computación en la nube, puede acceder a tanta o tan poca capacidad como necesite y escalar dinámicamente para satisfacer la demanda real, mientras paga solo por lo que usa.

En AWS, se pueden crear instancias de servidores, bases de datos, almacenamiento y componentes de aplicaciones de nivel superior en cuestión de segundos. Puede tratarlos como recursos temporales, libres de la inflexibilidad y las limitaciones de una infraestructura de TI fija y finita. Esto restablece la forma en que aborda la gestión de cambios, las pruebas, la confiabilidad y la planificación de la capacidad. Este cambio de enfoque fomenta la experimentación al introducir la capacidad en los procesos para fallar rápidamente e iterar rápidamente.

Capacidad global, disponible y escalable

Utilizando el infraestructura mundial de AWS, puede implementar su aplicación en la región de AWS que mejor se adapte a sus requisitos (p. ej., proximidad a sus usuarios finales, cumplimiento, restricciones de residencia de datos y costo).[3] Para las aplicaciones globales, puede reducir la latencia para los usuarios finales de todo el mundo mediante el uso de la red de entrega de contenido (CDN) de Amazon CloudFront. Esto también facilita mucho la operación de aplicaciones y bases de datos de producción en múltiples centros de datos para lograr una alta disponibilidad y tolerancia a fallas. La infraestructura global de AWS y la capacidad de aprovisionar la capacidad según sea necesario le permiten pensar de manera diferente sobre su infraestructura a medida que se expanden las demandas de sus aplicaciones y la amplitud de sus servicios.

Archivado

Servicios gestionados de nivel superior

Además de los recursos informáticos de Amazon Elastic Compute Cloud (Amazon EC2), también tiene acceso a un amplio conjunto de servicios de almacenamiento, bases de datos, análisis, aplicaciones e implementación. Debido a que estos servicios están disponibles instantáneamente para los desarrolladores, reducen la dependencia de las habilidades especializadas internas y permiten que las organizaciones entreguen nuevas soluciones más rápido. Los servicios de AWS que se administran pueden reducir la complejidad operativa y el costo. También están diseñados para ofrecer escalabilidad y alta disponibilidad, por lo que pueden reducir el riesgo de sus implementaciones.

Seguridad incorporada

En entornos de TI tradicionales, la auditoría de seguridad de la infraestructura puede ser un proceso periódico y manual. Por el contrario, la nube de AWS proporciona capacidades de gobernanza que permiten el monitoreo continuo de los cambios de configuración de sus recursos de TI. La seguridad en AWS es la máxima prioridad, lo que significa que se beneficia de los centros de datos y las arquitecturas de red que se construyen para cumplir con los requisitos de las organizaciones más sensibles a la seguridad.

Dado que los recursos de AWS se pueden programar mediante herramientas y API, puede formalizar e integrar su política de seguridad en el diseño de su infraestructura. Con la capacidad de activar entornos temporales, las pruebas de seguridad ahora pueden convertirse en parte de su canal de entrega continua. Finalmente, puede aprovechar una variedad de características nativas de seguridad y cifrado de AWS que pueden ayudarlo a lograr niveles más altos de protección de datos y cumplimiento.

Arquitectura por costo

La gestión de costes tradicional de las soluciones locales no suele estar estrechamente vinculada a la prestación de servicios. Cuando aprovisiona un entorno de computación en la nube, la optimización del costo es un inquilino de diseño fundamental para los arquitectos. Al seleccionar una solución, no solo debe centrarse en la arquitectura funcional y el conjunto de funciones, sino también en el perfil de costos de las soluciones que seleccione.

AWS proporciona facturación detallada, lo que le permite realizar un seguimiento de los costos asociados con todos los aspectos de sus soluciones. Hay una variedad de servicios para ayudarlo a administrar presupuestos, alertarlo sobre los costos incurridos y ayudarlo a optimizar el uso de recursos y los costos.

Archivado

Operaciones en AWS

Al operar servicios en AWS, existen varias categorías comunes de modelos operativos:

- Las aplicaciones que se migran, mantienen los modelos operativos tradicionales existentes, aprovechan la capacidad de administrar la infraestructura como código a través de las API, lo que permite procesos de construcción robustos y repetibles, lo que mejora la confiabilidad.
- Las soluciones que se refactorizan aprovechan niveles más altos de automatización de los procesos operativos como servicios de soporte, por ejemplo, AWS Auto Scaling y arquitecturas de autorreparación.
- Las soluciones rediseñadas y diseñadas para operaciones en la nube suelen estar totalmente automatizadas a través de procesos DevOps para canalización y gestión de entrega.

Apoyar estas transiciones no solo cambia las tecnologías utilizadas, sino también cambios culturales en la forma en que se gestionan los equipos operativos y de desarrollo.

AWS proporciona herramientas, procesos y prácticas recomendadas para respaldar la transición de las prácticas operativas a fin de maximizar los beneficios que pueden aprovechar de la computación en la nube.

Archivado

Criterios de diseño

La nube de AWS incluye muchos patrones de diseño y opciones arquitectónicas que puede aplicar a una amplia variedad de casos de uso. Algunos principios de diseño clave de la nube de AWS incluyen escalabilidad, recursos desechables, automatización, servicios administrados de acoplamiento flexible en lugar de servidores y opciones flexibles de almacenamiento de datos.

Escalabilidad

Los sistemas que se espera que crezcan con el tiempo deben construirse sobre una arquitectura escalable. Una arquitectura de este tipo puede admitir el crecimiento de usuarios, tráfico o tamaño de datos sin caídas en el rendimiento. Debe proporcionar esa escala de manera lineal donde agregar recursos adicionales da como resultado al menos un aumento proporcional en la capacidad de una carga adicional. El crecimiento debe introducir economías de escala y el costo debe seguir la misma dimensión que genera valor comercial a partir de ese sistema. Si bien la computación en la nube proporciona una capacidad bajo demanda virtualmente ilimitada, su diseño debe poder aprovechar esos recursos sin problemas.

En general, hay dos formas de escalar una arquitectura de TI: verticalmente y horizontalmente.

Escalar verticalmente

El escalado vertical se lleva a cabo mediante un aumento en las especificaciones de un recurso individual, como actualizar un servidor con un disco duro más grande o una CPU más rápida. Con Amazon EC2 puede detener una instancia y cambiar su tamaño a un tipo de instancia que tenga más RAM, CPU, E/S o capacidades de red. Esta forma de escalar eventualmente puede llegar a un límite, y no siempre es un enfoque rentable o de alta disponibilidad. Sin embargo, es muy fácil de implementar y puede ser suficiente para muchos casos de uso, especialmente a corto plazo.

Escalar horizontalmente

El escalado horizontal se lleva a cabo mediante un aumento en la cantidad de recursos, como agregar más discos duros a una matriz de almacenamiento o agregar más servidores para admitir una aplicación. Esta es una excelente manera de crear aplicaciones a escala de Internet que aprovechan la elasticidad de la computación en la nube. No todas las arquitecturas están diseñadas para distribuir su carga de trabajo a varios recursos, así que examinemos algunos escenarios posibles.

Archivado

Aplicaciones sin estado

Cuando los usuarios o los servicios interactúan con una aplicación, a menudo realizarán una serie de interacciones que forman una sesión. Una sesión son datos únicos para los usuarios que persisten entre solicitudes mientras usan la aplicación. Una aplicación sin estado es una aplicación que no necesita conocimiento de interacciones previas y no almacena información de sesión. Por ejemplo, una aplicación que, dada la misma entrada, proporciona la misma respuesta a cualquier usuario final, es una*aplicación sin estado*. Las aplicaciones sin estado pueden escalar horizontalmente porque cualquiera de los recursos informáticos disponibles (como las instancias EC2 y las funciones de AWS Lambda) pueden atender cualquier solicitud. Sin datos de sesión almacenados, simplemente puede agregar más recursos informáticos según sea necesario. Cuando esa capacidad ya no sea necesaria, puede terminar con seguridad esos recursos individuales, después de que se hayan agotado las tareas en ejecución. Esos recursos no necesitan estar al tanto de la presencia de sus pares; todo lo que se requiere es una forma de distribuirles la carga de trabajo.

Distribuya la carga a varios nodos

Para distribuir la carga de trabajo a varios nodos en su entorno, puede elegir un *empujar*o un*jalar*modelo.

Con un*empujar*modelo, puede usar Elastic Load Balancing (ELB) para distribuir una carga de trabajo. ELB enruta las solicitudes de aplicaciones entrantes a través de varias instancias de EC2. Al enrutar el tráfico, un balanceador de carga de red opera en la capa 4 del modelo de interconexión de sistemas abiertos (OSI) para manejar millones de solicitudes por segundo. Con la adopción de servicios basados en contenedores, también puede usar un balanceador de carga de aplicaciones. Un balanceador de carga de aplicaciones proporciona la capa 7 del modelo OSI y admite el enrutamiento de solicitudes basado en el contenido según el tráfico de la aplicación. De forma alternativa, puede utilizar Amazon Route 53 para implementar una operación por turnos de DNS. En este caso, las respuestas de DNS devuelven una dirección IP de una lista de hosts válidos por turnos. Si bien es fácil de implementar, este enfoque no siempre funciona bien con la elasticidad de la computación en la nube.

En lugar de una solución de equilibrio de carga, puede implementar una*jalar*modelo para cargas de trabajo asincrónicas basadas en eventos. En un modelo de extracción, las tareas que deben realizarse o los datos que deben procesarse se pueden almacenar como mensajes en una cola mediante Amazon Simple Queue Service (Amazon SQS) o como una solución de transmisión de datos.

Archivado

como Amazon Kinesis. Múltiples recursos informáticos pueden luego extraer y consumir esos mensajes, procesándolos de manera distribuida.

Componentes sin estado

En la práctica, la mayoría de las aplicaciones mantienen algún tipo de información de estado. Por ejemplo, las aplicaciones web necesitan rastrear si un usuario ha iniciado sesión para que se pueda presentar contenido personalizado basado en acciones anteriores. Un proceso automatizado de varios pasos también necesita realizar un seguimiento de la actividad anterior para decidir cuál debe ser su próxima acción. Todavía puede hacer que una parte de estas arquitecturas no tenga estado al no almacenar nada que deba persistir para más de una sola solicitud en el sistema de archivos local.

Por ejemplo, las aplicaciones web pueden usar cookies HTTP para almacenar información de la sesión (como elementos del carrito de compras) en la memoria caché del cliente web. El navegador devuelve esa información al servidor en cada solicitud posterior para que la aplicación no necesite almacenarla. Sin embargo, este enfoque tiene dos inconvenientes. En primer lugar, el contenido de las cookies HTTP se puede manipular en el lado del cliente, por lo que siempre debe tratarlo como datos no confiables que deben validarse. En segundo lugar, las cookies HTTP se transmiten con cada solicitud, lo que significa que debe mantener su tamaño al mínimo para evitar una latencia innecesaria.

Considere solo almacenar un identificador de sesión único en una cookie HTTP y almacenar información de sesión de usuario más detallada en el lado del servidor. La mayoría de las plataformas de programación proporcionan un mecanismo de gestión de sesión nativo que funciona de esta manera. Sin embargo, la información de la sesión del usuario a menudo se almacena en el sistema de archivos local de forma predeterminada, lo que da como resultado una arquitectura con estado. Una solución común a este problema es almacenar esta información en una base de datos. Amazon DynamoDB es una excelente opción debido a sus características de escalabilidad, alta disponibilidad y durabilidad. Para muchas plataformas, existen bibliotecas de reemplazo de código abierto que le permiten almacenar sesiones nativas en Amazon DynamoDB.[4]

Otros escenarios requieren el almacenamiento de archivos más grandes (como cargas de usuarios y resultados provisionales de procesos por lotes). Al colocar esos archivos en una capa de almacenamiento compartido como Amazon Simple Storage Service (Amazon S3) o Amazon Elastic File System (Amazon EFS), puede evitar la introducción de componentes con estado.

Finalmente, un flujo de trabajo complejo de varios pasos es otro ejemplo en el que debe realizar un seguimiento del estado actual de cada ejecución. Puede utilizar AWS Step Functions para almacenar de forma centralizada el historial de ejecución y hacer que estas cargas de trabajo no tengan estado.

Archivado

Componentes con estado

Inevitablemente, habrá capas de su arquitectura que no convertirá en componentes sin estado. Por definición, las bases de datos tienen estado. Para obtener más información, consulte el Sección de bases de datos . Además, muchas aplicaciones heredadas se diseñaron para ejecutarse en un solo servidor al depender de los recursos informáticos locales. Otros casos de uso pueden requerir que los dispositivos cliente mantengan una conexión a un servidor específico durante períodos prolongados. Por ejemplo, los juegos multijugador en tiempo real deben ofrecer a varios jugadores una vista uniforme del mundo del juego con una latencia muy baja. Esto es mucho más simple de lograr en una implementación no distribuida donde los participantes están conectados al mismo servidor.

Es posible que aún pueda escalar esos componentes horizontalmente distribuyendo la carga a varios nodos con afinidad de sesión. En este modelo, vincula todas las transacciones de una sesión a un recurso informático específico. Pero, este modelo tiene algunas limitaciones. Las sesiones existentes no se benefician directamente de la introducción de nodos de cómputo recién lanzados. Más importante aún, no se puede garantizar la afinidad de la sesión. Por ejemplo, cuando un nodo finaliza o deja de estar disponible, los usuarios vinculados a él se desconectarán y experimentarán una pérdida de datos específicos de la sesión, que es cualquier cosa que no esté almacenada en un recurso compartido como Amazon S3, Amazon EFS o un base de datos.

Implementar afinidad de sesión

Para el tráfico HTTP y HTTPS, puede utilizar *sesiones pegajosas*característica de un balanceador de carga de aplicaciones para vincular la sesión de un usuario a una instancia específica.[5]Con esta función, un Balanceador de carga de aplicaciones intentará usar el mismo servidor para ese usuario durante la sesión.

Otra opción, si controla el código que se ejecuta en el cliente, es usar el equilibrio de carga del lado del cliente. Esto agrega complejidad adicional, pero puede ser útil en escenarios donde un balanceador de carga no cumple con sus requisitos. Por ejemplo, es posible que esté utilizando un protocolo que no es compatible con ELB, o que necesite un control total sobre cómo se asignan los usuarios a los servidores (por ejemplo, en un escenario de juego, es posible que deba asegurarse de que los participantes del juego coincidan y se conecten a el mismo servidor). En este modelo, los clientes necesitan una forma de descubrir puntos finales de servidor válidos para conectarse directamente. Puede usar DNS para eso, o puede crear una API de descubrimiento simple para proporcionar esa información al software que se ejecuta en el cliente. En ausencia de un balanceador de carga, el mecanismo de verificación de estado también debe implementarse en el lado del cliente.

Archivado

Procesamiento distribuido

Los casos de uso que involucran el procesamiento de grandes cantidades de datos, cualquier cosa que no pueda ser manejada por un solo recurso informático de manera oportuna, requieren un enfoque de procesamiento distribuido. Al dividir una tarea y sus datos en muchos pequeños fragmentos de trabajo, puede ejecutarlos en paralelo en un conjunto de recursos informáticos.

Implementar procesamiento distribuido

Los trabajos por lotes sin conexión se pueden escalar horizontalmente mediante el uso de motores de procesamiento de datos distribuidos, como AWS Batch, AWS Glue y Apache Hadoop. En AWS, puede usar Amazon EMR para ejecutar cargas de trabajo de Hadoop sobre una flota de instancias EC2 sin la complejidad operativa. Para el procesamiento en tiempo real de los datos de transmisión, Amazon Kinesis divide los datos en varios fragmentos que luego pueden consumir varios recursos de Amazon EC2 o AWS Lambda para lograr la escalabilidad.

Para obtener más información sobre estos tipos de cargas de trabajo, consulte el Opciones de análisis de Big Data en AWS[6] y Principios básicos de IoT libros blancos.[7]

Recursos desechables en lugar de servidores fijos

En un entorno de infraestructura tradicional, debe trabajar con recursos fijos debido al costo inicial y al tiempo de espera de la introducción de nuevo hardware. Esto impulsa prácticas como iniciar sesión manualmente en servidores para configurar software o solucionar problemas, codificar direcciones IP, ejecutar pruebas o procesar trabajos secuencialmente.

Al diseñar para AWS, puede aprovechar la naturaleza de aprovisionamiento dinámico de la computación en la nube. Puede pensar en servidores y otros componentes como recursos temporales. Puede iniciar tantos como necesite y usarlos solo durante el tiempo que los necesite.

Otro problema con los servidores fijos de larga ejecución es *desviación de configuración*. Los cambios y parches de software aplicados a lo largo del tiempo pueden generar configuraciones no probadas y heterogéneas en diferentes entornos. Puede resolver este problema con un patrón de infraestructura inmutable. Con este enfoque, un servidor, una vez lanzado, nunca se actualiza. En su lugar, cuando hay un problema o se necesita una actualización, el servidor problemático se reemplaza por un servidor nuevo que tiene la configuración más reciente. Esto permite que los recursos estén siempre en un estado consistente (y probado), y hace que las reversiones sean más fáciles de realizar. Esto se admite más fácilmente con arquitecturas sin estado.

Archivado

Creación de instancias de recursos informáticos

Ya sea que esté implementando un nuevo entorno para realizar pruebas o aumentando la capacidad de un sistema existente para hacer frente a una carga adicional, no desea configurar manualmente nuevos recursos con su configuración y código. Es importante que haga de este un proceso automatizado y repetible que evite largos plazos de entrega y que no sea propenso a errores humanos. Hay algunos métodos para lograr esto.

arranque

Cuando lanza un recurso de AWS, como una instancia EC2 o una instancia de base de datos de Amazon Relational Database Service (Amazon RDS), comienza con una configuración predeterminada. Luego puede ejecutar acciones de arranque automáticas, que son secuencias de comandos que instalan software o copian datos para llevar ese recurso a un estado particular. Puede parametrizar detalles de configuración que varían entre diferentes entornos (como producción o prueba) para que pueda reutilizar los mismos scripts sin modificaciones.

Puede configurar nuevas instancias EC2 con el usuario scripts de datos, directivas cloud-init.[8] Puede usar scripts simples y herramientas de administración de configuración como Chef o Puppet. Además, con scripts personalizados y las API de AWS, o con el soporte de AWS CloudFormation para Recursos personalizados respaldados por AWS Lambda, puede escribir una lógica de aprovisionamiento que actúe en casi cualquier recurso de AWS.[9]

Imágenes doradas

Ciertos tipos de recursos de AWS, como instancias EC2, instancias de base de datos de Amazon RDS y volúmenes de Amazon Elastic Block Store (Amazon EBS), se pueden lanzar desde un *imagen dorada*, que es una instantánea de un estado particular de ese recurso. En comparación con el enfoque de arranque, una imagen dorada da como resultado tiempos de inicio más rápidos y elimina las dependencias de los servicios de configuración o repositorios de terceros. Esto es importante en entornos de escalado automático en los que desea poder lanzar recursos adicionales de manera rápida y confiable como respuesta a los cambios en la demanda.

Puede personalizar una instancia EC2 y luego guardar su configuración creando un Imagen de máquina de Amazon (AMI) .[10] Puede lanzar tantas instancias desde la AMI como necesite, y todas incluirán esas personalizaciones. Cada vez que desee cambiar su configuración, debe crear una nueva imagen dorada, por lo que debe tener una convención de control de versiones para administrar sus imágenes doradas a lo largo del tiempo. Le recomendamos que utilice un script para crear el arranque para las instancias EC2 que utiliza para crear

Archivado

sus AMI. Esto le brinda un método flexible para probar y modificar esas imágenes a lo largo del tiempo.

Como alternativa, si tiene un entorno virtualizado local existente, puede usar VM Import/Export de AWS para convertir una variedad de formatos de virtualización a una AMI. También puede buscar y utilizar AMI compartidas prefabricadas proporcionadas por AWS o por terceros en AWS Marketplace.

Si bien las imágenes doradas se usan más comúnmente cuando lanza nuevas instancias EC2, también se pueden aplicar a recursos como instancias de base de datos de Amazon RDS o volúmenes de Amazon EBS. Por ejemplo, cuando lanza un nuevo entorno de prueba, es posible que desee rellenar previamente su base de datos creando una instancia de una instantánea específica de Amazon RDS, en lugar de importar los datos desde un extenso script SQL.

Contenedores

Otra opción popular entre los desarrolladores es Docker, una tecnología de código abierto que permite crear e implementar aplicaciones distribuidas dentro de contenedores de software. Docker le permite empaquetar una pieza de software en una imagen de Docker, que es una unidad estandarizada para el desarrollo de software, que contiene todo lo que el software necesita para ejecutarse: código, tiempo de ejecución, herramientas del sistema, bibliotecas del sistema, etc. AWS Elastic Beanstalk, Amazon Elastic Container Service (Amazon ECS) y AWS Fargate le permiten implementar y administrar varios contenedores en un clúster de instancias ECS. Puede crear imágenes doradas de Docker y usar ECS Container Registry para administrarlas.

Un entorno de contenedor alternativo es Kubernetes y Amazon Elastic Container Service para Kubernetes (Amazon EKS). Con Kubernetes y Amazon EKS, puede implementar, administrar y escalar fácilmente aplicaciones en contenedores.

Híbrido

También puede usar una combinación de los dos enfoques: algunas partes de la configuración se capturan en una imagen dorada, mientras que otras se configuran dinámicamente a través de una acción de arranque.

Los elementos que no cambian con frecuencia o que introducen dependencias externas generalmente serán parte de su imagen dorada. Un ejemplo de un buen candidato es el software de su servidor web que, de lo contrario, tendría que ser descargado por un repositorio de terceros cada vez que inicie una instancia.

Archivado

Los elementos que cambian con frecuencia o difieren entre los diversos entornos se pueden configurar dinámicamente mediante acciones de arranque. Por ejemplo, si implementa nuevas versiones de su aplicación con frecuencia, la creación de una nueva AMI para cada versión de la aplicación puede resultar poco práctica. Tampoco desea codificar la configuración del nombre de host de la base de datos en su AMI porque sería diferente entre los entornos de prueba y producción. Los datos de usuario o las etiquetas le permiten usar AMI más genéricas que se pueden modificar en el momento del lanzamiento. Por ejemplo, si ejecuta servidores web para varias pequeñas empresas, todas pueden usar la misma AMI y recuperar su contenido desde una ubicación de depósito S3 que especifique en los datos de usuario en el lanzamiento.

AWS Elastic Beanstalk sigue el modelo híbrido. Proporciona entornos de tiempo de ejecución preconfigurados, cada unoiniciado desde su propia AMI [11]—pero le permite ejecutar acciones de arranque a través deArchivos de configuración .ebextensions [12]y configurar variables ambientales para parametrizar las diferencias ambientales.

Para una discusión más detallada de las diferentes formas en que puede administrar las implementaciones de nuevos recursos, consulte laDescripción general de las opciones de implementación en AWS [13]yAdministración de su infraestructura de AWS a escala libros blancos.[14]

Infraestructura como código

La aplicación de los principios que hemos discutido no tiene que limitarse al nivel de recursos individuales. Debido a que los activos de AWS son programables, puede aplicar técnicas, prácticas y herramientas del desarrollo de software para hacer que toda su infraestructura sea reutilizable, mantenible, extensible y comprobable. Para obtener más información, consulte elInfraestructura como código papel blanco.[15]

Plantillas de AWS CloudFormationle ofrece una manera fácil de crear y administrar una colección de recursos de AWS relacionados, y de aprovisionarlos y actualizarlos de manera ordenada y predecible. Puede describir los recursos de AWS y las dependencias asociadas o los parámetros de tiempo de ejecución necesarios para ejecutar su aplicación. Sus plantillas de CloudFormation pueden convivir con su aplicación en su repositorio de control de versiones, lo que le permite reutilizar arquitecturas y clonar entornos de producción de manera confiable para realizar pruebas.

Automatización

En una infraestructura de TI tradicional, a menudo tiene que reaccionar manualmente a una variedad de eventos. Al implementar en AWS, existe una oportunidad para la automatización, de modo que usted

Archivado

mejore tanto la estabilidad de su sistema como la eficiencia de su organización. Considere la posibilidad de introducir uno o más de estos tipos de automatización en la arquitectura de su aplicación para garantizar una mayor resiliencia, escalabilidad y rendimiento.

Gestión e implementación sin servidor

Cuando adopta patrones sin servidor, el enfoque operativo está en la automatización de la canalización de implementación. AWS administra los servicios subyacentes, la escala y la disponibilidad. AWS CodePipeline, AWS CodeBuild y AWS CodeDeploy admiten la automatización de la implementación de estos procesos.[dieciséis]

Gestión e implementación de infraestructura

Beanstalk elástico de AWS : puede usar este servicio para implementar y escalar aplicaciones y servicios web desarrollados con Java, .NET, PHP, Node.js, Python, Ruby, Go y Docker en servidores familiares como Apache, Nginx, Passenger e IIS.[17]Los desarrolladores simplemente pueden cargar el código de aplicación y el servicio maneja automáticamente todos los detalles, como el aprovisionamiento de recursos, el equilibrio de carga, el escalado automático y la supervisión.

Recuperación automática de Amazon EC2 : puede crear una alarma de Amazon CloudWatch que supervise una instancia EC2 y la recupere automáticamente si se daña.[18]Una instancia recuperada es idéntica a la instancia original, incluida la ID de la instancia, las direcciones IP privadas, las direcciones IP elásticas y todos los metadatos de la instancia. Sin embargo, esta característica solo está disponible para configuraciones de instancias aplicables. Consulte la documentación de Amazon EC2 para obtener una descripción actualizada de esas condiciones previas. Además, durante la recuperación de la instancia, la instancia se mueve a través de un reinicio de la instancia y se pierde cualquier dato que esté en la memoria.

Administrador de sistemas de AWS : puede recopilar automáticamente el inventario de software, aplicar parches del sistema operativo, crear una imagen del sistema para configurar los sistemas operativos Windows y Linux y ejecutar comandos arbitrarios. El aprovisionamiento de estos servicios simplifica el modelo operativo y asegura la configuración óptima del entorno.[19]

Escalado automático : puede mantener la disponibilidad de la aplicación y aumentar o reducir automáticamente la capacidad de Amazon EC2, Amazon DynamoDB, Amazon ECS, Amazon Elastic Container Service para Kubernetes (Amazon EKS) según las condiciones que defina.[20]Puede usar Auto Scaling para ayudar a asegurarse de que está ejecutando la cantidad deseada de instancias EC2 en buen estado en varias zonas de disponibilidad. Auto Scaling también puede aumentar automáticamente la cantidad de instancias EC2 durante picos de demanda para

Archivado

mantenga el rendimiento y reduzca la capacidad durante los períodos menos ocupados para optimizar los costos.

Alarmas y Eventos

Alarmas de Amazon CloudWatch : puede crear una alarma de CloudWatch que envíe un mensaje de Amazon Simple Notification Service (Amazon SNS) cuando una métrica en particular supere un umbral específico durante una cantidad específica de períodos.[21]Esos mensajes de Amazon SNS pueden iniciar automáticamente la ejecución de una función Lambda suscrita, poner en cola un mensaje de notificación en una cola de Amazon SQS o realizar una solicitud POST a un punto de enlace HTTP o HTTPS.

Eventos de Amazon CloudWatch : ofrece un flujo casi en tiempo real de eventos del sistema que describen cambios en los recursos de AWS.[22]Con reglas simples, puede enrutar cada tipo de evento a uno o más destinos, como funciones de Lambda, flujos de Kinesis y temas de SNS.

Eventos programados de AWS Lambda : puede crear una función Lambda y configurar AWS Lambda para ejecutarla en un horario regular.[23]

Automatizaciones de seguridad de AWS WAF : AWS WAF es un firewall de aplicaciones web que le permite crear reglas personalizadas específicas de la aplicación que bloqueen patrones de ataque comunes que pueden afectar la disponibilidad de la aplicación, comprometer la seguridad o consumir recursos excesivos. Puede administrar AWS WAF completamente a través de las API, lo que facilita la automatización de la seguridad, lo que permite una rápida propagación de reglas y una rápida respuesta a incidentes.[24]

Bajo acoplamiento

A medida que aumenta la complejidad de la aplicación, un atributo deseable de un sistema de TI es que se puede dividir en componentes más pequeños y poco acoplados. Esto significa que los sistemas de TI deben diseñarse de manera que reduzcan las interdependencias: un cambio o una falla en un componente no debe afectar a otros componentes.

Interfaces bien definidas

Una forma de reducir las interdependencias en un sistema es permitir que los diversos componentes interactúen entre sí solo a través de interfaces específicas e independientes de la tecnología, como las API RESTful. De esa manera, los detalles de implementación técnica se ocultan para que los equipos puedan

Archivado

modificar la implementación subyacente sin afectar a otros componentes. Siempre que esas interfaces mantengan la compatibilidad con versiones anteriores, las implementaciones de componentes diferentes se desacoplarán. Este patrón de diseño granular se conoce comúnmente como arquitectura de microservicios.

Puerta de enlace API de Amazones un servicio completamente administrado que facilita a los desarrolladores la creación, publicación, mantenimiento, monitoreo y protección de API a cualquier escala. Maneja todas las tareas relacionadas con la aceptación y el procesamiento de hasta cientos de miles de llamadas API simultáneas, incluida la gestión del tráfico, la autorización y el control de acceso, la supervisión y la gestión de versiones de la API.

Descubrimiento de servicios

Las aplicaciones que se implementan como un conjunto de servicios más pequeños dependen de la capacidad de esos servicios para interactuar entre sí. Debido a que cada uno de esos servicios puede ejecutarse en múltiples recursos informáticos, debe haber una manera de abordar cada servicio. Por ejemplo, en una infraestructura tradicional, si su servicio web front-end necesita conectarse con su servicio web back-end, podría codificar la dirección IP del recurso informático donde se estaba ejecutando este servicio. Si bien este enfoque aún puede funcionar en la computación en la nube, si esos servicios están destinados a estar acoplados de manera flexible, deberían poder consumirse sin conocimiento previo de los detalles de la topología de su red. Además de ocultar la complejidad, esto también permite que los detalles de la infraestructura cambien en cualquier momento. El acoplamiento flexible es un elemento crucial si desea aprovechar la elasticidad de la computación en la nube, donde se pueden iniciar o finalizar nuevos recursos en cualquier momento. Para lograrlo, necesitará alguna forma de implementar el descubrimiento de servicios.

Implementar el descubrimiento de servicios

Para un servicio alojado en Amazon EC2, una forma sencilla de lograr el descubrimiento del servicio es a través de Elastic Load Balancing (ELB). Debido a que cada equilibrador de carga tiene su propio nombre de host, puede consumir un servicio a través de un punto final estable. Esto se puede combinar con DNS y zonas privadas de Amazon Route 53, de modo que el punto final del balanceador de carga en particular se pueda abstraer y modificar en cualquier momento.

Otra opción es utilizar un método de descubrimiento y registro de servicios para permitir la recuperación de las direcciones IP del punto final y el número de puerto de cualquier servicio. Debido a que el descubrimiento de servicios se convierte en el vínculo entre los componentes, es importante que sea altamente disponible y confiable. Si no se utilizan balanceadores de carga, el descubrimiento de servicios también debe

Archivado

Permitir opciones como controles de salud. Amazon Route 53 admite la asignación automática de nombres para facilitar el aprovisionamiento de instancias para microservicios. La asignación automática de nombres le permite crear automáticamente registros DNS en función de una configuración que defina. Otros ejemplos de implementaciones incluyen soluciones personalizadas que utilizan una combinación de etiquetas, una base de datos de alta disponibilidad, scripts personalizados que llaman a las API de AWS o herramientas de código abierto como Netflix Eureka, Airbnb Synapse o HashiCorp Consul.

Integración asíncrona

La integración asíncrona es otra forma de acoplamiento flexible entre servicios. Este modelo es adecuado para cualquier interacción que no necesite una respuesta inmediata y donde será suficiente un reconocimiento de que se ha registrado una solicitud. Se trata de un componente que genera eventos y otro que los consume. Los dos componentes no se integran a través de una interacción punto a punto directa, sino a través de una capa de almacenamiento duradera intermedia, como una cola de SQS o una plataforma de transmisión de datos como Amazon Kinesis, eventos de Lambda en cascada, AWS Step Functions o Amazon Simple Workflow. Servicio.

educar

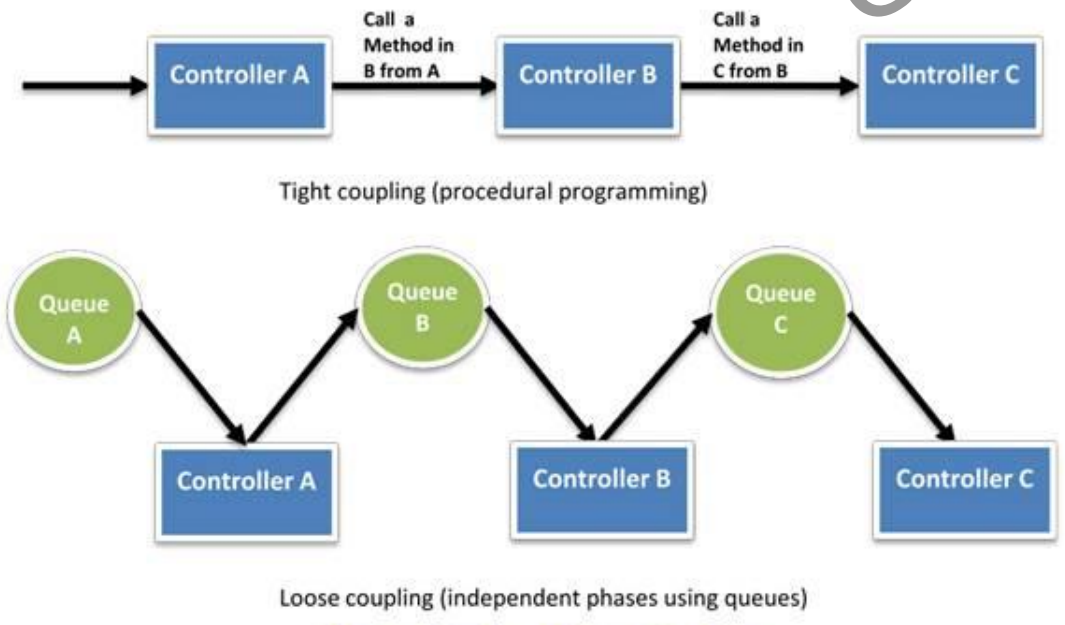

Figura 1: Acoplamiento apretado y flojo

Este enfoque desacopla los dos componentes e introduce resiliencia adicional. Entonces, por ejemplo, si falla un proceso que está leyendo mensajes de la cola, aún se pueden agregar mensajes a la cola y procesarlos cuando el sistema se recupere. También le permite proteger un servicio de back-end menos escalable de los picos de front-end y encontrar el adecuado

compensación entre el costo y el retraso en el procesamiento. Por ejemplo, puede decidir que no necesita escalar su base de datos para adaptarse a un pico ocasional de consultas de escritura, siempre que finalmente procese esas consultas de forma asíncrona con algún retraso. Finalmente, al eliminar las operaciones lentas de las rutas de solicitud interactivas, también puede mejorar la experiencia del usuario final.

Los ejemplos de integración asíncrona incluyen:

- Una aplicación front-end inserta trabajos en un sistema de cola como Amazon SQS. Un sistema de back-end recupera esos trabajos y los procesa a su propio ritmo.
- Una API genera eventos y los inserta en flujos de Kinesis. Una aplicación de back-end procesa estos eventos en lotes para crear datos agregados de series de tiempo almacenados en una base de datos.
- Múltiples sistemas heterogéneos utilizan AWS Step Functions para comunicar el flujo de trabajo entre ellos sin interactuar directamente entre sí.
- Las funciones de Lambda pueden consumir eventos de una cantidad de fuentes de AWS, como flujos de actualización de Amazon DynamoDB y notificaciones de eventos de Amazon S3. No tiene que preocuparse por implementar una cola u otro método de integración asincrónica porque Lambda se encarga de esto por usted.

Mejores prácticas de sistemas distribuidos

Otra forma de aumentar el acoplamiento flexible es crear aplicaciones que manejen fallas de componentes de manera elegante. Puede identificar formas de reducir el impacto para sus usuarios finales y aumentar su capacidad para progresar en sus procedimientos fuera de línea, incluso en el caso de que falle algún componente.

Fracaso elegante en la práctica

Una solicitud que falla se puede volver a intentar con un exponencialEstrategia de backoff y Jitter , o se puede almacenar en una cola para su posterior procesamiento.[25]Para las interfaces frontales, podría ser posible proporcionar contenido alternativo o en caché en lugar de fallar por completo cuando, por ejemplo, su servidor de base de datos no esté disponible. La función de conmutación por error de DNS de Amazon Route 53 también le brinda la capacidad de monitorear su sitio web y enrutar automáticamente a sus visitantes a un sitio de respaldo si su sitio principal no está disponible. Puede alojar su sitio de respaldo como un sitio web estático en Amazon S3 o como un entorno dinámico separado.

Archivado

Servicios, no servidores

Desarrollar, administrar y operar aplicaciones, especialmente a escala, requiere una amplia variedad de componentes tecnológicos subyacentes. Con la infraestructura de TI tradicional, las empresas tendrían que construir y operar todos esos componentes.

AWS ofrece un amplio conjunto de servicios de cómputo, almacenamiento, bases de datos, análisis, aplicaciones e implementación que ayudan a las organizaciones a moverse más rápido y reducir los costos de TI.

Las arquitecturas que no aprovechan esa amplitud (p. ej., si solo usan Amazon EC2) podrían no aprovechar al máximo la computación en la nube y perder la oportunidad de aumentar la productividad y la eficiencia operativa de los desarrolladores.

Servicios gestionados

Los servicios administrados de AWS proporcionan componentes básicos que los desarrolladores pueden consumir para potenciar sus aplicaciones. Estos servicios administrados incluyen bases de datos, aprendizaje automático, análisis, colas, búsqueda, correo electrónico, notificaciones y más. Por ejemplo, con Amazon SQS puede descargar la carga administrativa de operar y escalar un clúster de mensajería de alta disponibilidad, mientras paga un precio bajo solo por lo que usa. Amazon SQS también es inherentemente escalable y confiable. Lo mismo se aplica a Amazon S3, que le permite almacenar tantos datos como desee y acceder a esos datos cuando los necesite, sin tener que pensar en la capacidad, las configuraciones del disco duro, la replicación u otros problemas relacionados.

Otros ejemplos de los servicios administrados que impulsan sus aplicaciones incluyen:[26]

- Amazon CloudFront para la entrega de contenido
- ELB para equilibrio de carga
- Amazon DynamoDB para bases de datos NoSQL
- Amazon CloudSearch para cargas de trabajo de búsqueda
- Amazon Elastic Transcoder para codificación de video
- Amazon Simple Email Service (Amazon SES) para enviar y recibir correos electrónicos[27]

Archivado

Arquitecturas sin servidor

Las arquitecturas sin servidor pueden reducir la complejidad operativa de las aplicaciones en ejecución. Es posible crear servicios sincrónicos y basados en eventos para dispositivos móviles, web, análisis, lógica comercial de CDN e IoT sin administrar ninguna infraestructura de servidor. Estas arquitecturas pueden reducir los costos porque no tiene que administrar ni pagar por servidores infrautilizados, ni aprovisionar infraestructura redundante para implementar alta disponibilidad.

Por ejemplo, puede cargar su código en el servicio informático de AWS Lambda y el servicio puede ejecutar el código en su nombre mediante la infraestructura de AWS. Con AWS Lambda, se le cobra por cada 100 ms que se ejecuta su código y la cantidad de veces que se activa su código. Al utilizar Amazon API Gateway, puede desarrollar API sincrónicas virtualmente infinitamente escalables con la tecnología de AWS Lambda. Cuando se combina con Amazon S3 para servir activos de contenido estático, este patrón puede ofrecer una aplicación web completa. Para más detalles sobre este tipo de arquitecturas, ver el Arquitecturas de varios niveles sin servidor de AWS papel blanco.[28]

Cuando se trata de aplicaciones móviles y web, puede usar Amazon Cognito para no tener que administrar una solución de back-end para manejar la autenticación de usuarios, el estado de la red, el almacenamiento y la sincronización. Amazon Cognito genera identificadores únicos para sus usuarios.

Se puede hacer referencia a esos identificadores en sus políticas de acceso para habilitar o restringir el acceso a otros recursos de AWS por usuario. Amazon Cognito proporciona credenciales temporales de AWS a sus usuarios, lo que permite que la aplicación móvil que se ejecuta en el dispositivo interactúe directamente con los servicios de AWS protegidos por AWS Identity and Access Management (IAM). Por ejemplo, con IAM puede restringir el acceso a una carpeta en un depósito S3 a un usuario final en particular.

Para las aplicaciones de IoT, las organizaciones tradicionalmente han tenido que aprovisionar, operar, escalar y mantener sus propios servidores como puertas de enlace de dispositivos para manejar la comunicación entre los dispositivos conectados y sus servicios. AWS IoT proporciona una puerta de enlace para dispositivos completamente administrada que escala automáticamente con su uso sin sobrecarga operativa.

Las arquitecturas sin servidor también han hecho posible ejecutar servicios receptivos en ubicaciones de borde. AWS Lambda@Edge le permite ejecutar funciones de Lambda en ubicaciones de borde de Amazon CloudFront en respuesta a eventos de CloudFront. Esto permite patrones para baja latencia

Archivado

soluciones y la introducción de funcionalidad sin cambiar las aplicaciones subyacentes.[29]

Para el análisis de datos, la administración de consultas en grandes volúmenes de datos generalmente requiere que administre una infraestructura compleja. Amazon Athena es un servicio de consulta interactivo que le facilita el análisis de datos en Amazon S3 mediante SQL estándar. Athena no tiene servidor, por lo que no hay infraestructura que administrar y solo paga por las consultas que ejecuta.

bases de datos

Con la infraestructura de TI tradicional, las organizaciones a menudo se limitan a las tecnologías de base de datos y almacenamiento que pueden utilizar. Puede haber restricciones basadas en los costos de licencia y la capacidad de admitir diversos motores de bases de datos. En AWS, estas restricciones se eliminan mediante servicios de base de datos administrados que ofrecen rendimiento empresarial a un costo de código abierto. Como resultado, no es raro que las aplicaciones se ejecuten sobre una capa de datos políglota que elija la tecnología adecuada para cada carga de trabajo.

Elija la tecnología de base de datos adecuada para cada carga de trabajo

Estas preguntas pueden ayudarlo a tomar decisiones sobre qué soluciones incluir en su arquitectura:

- ¿Se trata de una carga de trabajo de muchas lecturas, muchas escrituras o equilibrada? ¿Cuántas lecturas y escrituras por segundo vas a necesitar? ¿Cómo cambiarán esos valores si aumenta el número de usuarios?
- ¿Cuántos datos necesitará almacenar y durante cuánto tiempo? ¿Qué tan rápido crecerá esto? ¿Hay un límite superior en un futuro próximo? ¿Cuál es el tamaño de cada objeto (promedio, mínimo, máximo)? ¿Cómo se accederá a estos objetos?
- ¿Cuáles son los requisitos en términos de durabilidad de los datos? ¿Será este almacén de datos su "fuente de la verdad"?
- ¿Cuáles son sus requisitos de latencia? ¿Cuántos usuarios simultáneos necesita admitir?
- ¿Cuál es su modelo de datos y cómo va a consultar los datos? ¿Sus consultas son de naturaleza relacional (p. ej., ÚNASE entre varias tablas)? ¿Podría desnormalizar su esquema para crear estructuras de datos más planas que sean más fáciles de escalar?

Archivado

- ¿Qué tipo de funcionalidad necesita? ¿Necesita fuertes controles de integridad o busca más flexibilidad (p. ej., almacenes de datos sin esquema)? ¿Necesita funciones sofisticadas de generación de informes o de búsqueda? ¿Sus desarrolladores están más familiarizados con las bases de datos relacionales que con NoSQL?
- ¿Cuáles son los costos de licencia de tecnología de base de datos asociados? ¿Estos costos consideran la inversión en desarrollo de aplicaciones, el almacenamiento y los costos de uso a lo largo del tiempo? ¿El modelo de licencia respalda el crecimiento proyectado? ¿Podría utilizar motores de bases de datos nativos de la nube como Amazon Aurora para obtener la simplicidad y la rentabilidad de las bases de datos de código abierto?

Bases de datos relacionales

Las bases de datos relacionales (también conocidas como bases de datos RDBS o SQL) normalizan los datos en estructuras tabulares bien definidas conocidas como tablas, que constan de filas y columnas. Proporcionan un potente lenguaje de consulta, capacidades de indexación flexibles, sólidos controles de integridad y la capacidad de combinar datos de varias tablas de forma rápida y eficiente. Amazon RDS facilita la configuración, el funcionamiento y el escalado de una base de datos relacional en la nube con compatibilidad con muchos motores de bases de datos conocidos.

Escalabilidad

Las bases de datos relacionales pueden escalar verticalmente mediante la actualización a una instancia de base de datos de Amazon RDS más grande o agregando más almacenamiento y más rápido. Además, considere usar Amazon Aurora, que es un motor de base de datos diseñado para ofrecer un rendimiento mucho mayor en comparación con MySQL estándar que se ejecuta en el mismo hardware para aplicaciones de lectura intensiva, también puede escalar horizontalmente más allá de las limitaciones de capacidad de una única instancia de base de datos mediante la creación de una o más réplicas de lectura.

Las réplicas de lectura son instancias de bases de datos independientes que se replican de forma asíncrona. Como resultado, están sujetos a retrasos en la replicación y es posible que falten algunas de las transacciones más recientes. Los diseñadores de aplicaciones deben considerar qué consultas tienen tolerancia a datos ligeramente obsoletos. Esas consultas se pueden ejecutar en una réplica de lectura, mientras que el resto debe ejecutarse en el nodo principal. Las réplicas de lectura tampoco pueden aceptar consultas de escritura.

Las cargas de trabajo de bases de datos relacionales que necesitan escalar su capacidad de escritura más allá de las limitaciones de una sola instancia de base de datos requieren un enfoque diferente llamado*partición de datos* o *fragmentación*. Con este modelo, los datos se dividen en varios esquemas de base de datos, cada uno

Archivado

ejecutándose en su propia instancia de base de datos primaria autónoma. Aunque Amazon RDS elimina la sobrecarga operativa de ejecutar esas instancias, la fragmentación introduce cierta complejidad en la aplicación. La capa de acceso a datos de la aplicación debe modificarse para tener conocimiento de cómo se dividen los datos para que pueda dirigir las consultas a la instancia correcta. Además, los cambios de esquema se deben realizar en varios esquemas de base de datos, por lo que vale la pena invertir un poco de esfuerzo para automatizar este proceso.

Alta disponibilidad

Para cualquier base de datos relacional de producción, recomendamos utilizar la función de implementación Multi-AZ de Amazon RDS, que crea una instancia en espera replicada sincrónicamente en una zona de disponibilidad diferente. En caso de falla del nodo principal, Amazon RDS realiza una conmutación por error automática al nodo en espera sin necesidad de una intervención administrativa manual. Cuando se realiza una conmutación por error, hay un breve período durante el cual no se puede acceder al nodo principal. Se pueden diseñar aplicaciones resilientes paraFracaso agradado y ofrecer una funcionalidad reducida, como el modo de solo lectura mediante el uso de réplicas de lectura. Amazon Aurora proporciona capacidad multimaestro para permitir que las lecturas y escrituras se escalen en las zonas de disponibilidad y también admite la replicación entre regiones.

Antipatrones

Si su aplicación indexa y consulta principalmente datos sin necesidad de uniones o transacciones complejas, especialmente si espera un rendimiento de escritura más allá de las limitaciones de una sola instancia, considere una base de datos NoSQL en su lugar. Si tiene archivos binarios grandes (audio, video e imagen), será más eficiente almacenar los archivos reales en Amazon S3 y solo mantener los metadatos de los archivos en su base de datos.

Para conocer las mejores prácticas de bases de datos relacionales más detalladas, consulte elDocumentación de Amazon RDS.

Archivado

Bases de datos NoSQL

Las bases de datos NoSQL intercambian algunas de las capacidades de consulta y transacción de las bases de datos relacionales por un modelo de datos más flexible que escala horizontalmente sin problemas. Las bases de datos NoSQL utilizan una variedad de modelos de datos, incluidos gráficos, pares clave-valor y documentos JSON, y son ampliamente reconocidas por su facilidad de desarrollo, rendimiento escalable, alta disponibilidad y resiliencia. Amazon DynamoDB es una solución rápida y flexibleBase de datos NoSQL servicio para aplicaciones que necesitan una latencia constante de milisegundos de un solo dígito a cualquier escala.[31]Es una base de datos en la nube completamente administrada y admite modelos de almacenamiento de documentos y valores clave.

Escalabilidad

Los motores de base de datos NoSQL normalmente realizarán la partición y la replicación de datos para escalar tanto las lecturas como las escrituras de forma horizontal. Lo hacen de forma transparente y no necesitan la lógica de partición de datos implementada en la capa de acceso a datos de su aplicación. Amazon DynamoDB, en particular, administra la partición de tablas automáticamente, agregando nuevas particiones a medida que su tabla crece en tamaño o cuando cambia la capacidad de lectura y escritura. Amazon DynamoDB Accelerator (DAX) es una caché en memoria administrada y de alta disponibilidad para que DynamoDB aproveche mejoras significativas en el rendimiento.[32]

Para obtener información sobre cómo puede aprovechar al máximo la escalabilidad de Amazon DynamoDB al diseñar su aplicación, consultePrácticas recomendadas para DynamoDB .[33]

Alta disponibilidad

Amazon DynamoDB replica sincrónicamente los datos en tres instalaciones en una región de AWS, lo que proporciona tolerancia a fallas en caso de falla del servidor o interrupción de la zona de disponibilidad. Amazon DynamoDB también es compatible con tablas globales para proporcionar una base de datos multimaestro, de varias regiones y completamente administrada que proporciona un rendimiento de lectura y escritura local rápido para aplicaciones globales de escala masiva. Las tablas globales se replican en las regiones de AWS seleccionadas.

Antipatrones

Si su esquema no se puede desnormalizar y su aplicación requiere uniones o transacciones complejas, considere una base de datos relacional en su lugar. Si tiene archivos binarios grandes (audio, video e imagen), considere almacenar los archivos en Amazon S3 y almacenar los metadatos de los archivos en su base de datos.

Archivado

Para obtener instrucciones sobre cómo migrar de una base de datos relacional a DynamoDB, o sobre cómo evaluar qué cargas de trabajo migrar, consulte elPrácticas recomendadas para migrar de RDBMS a DynamoDB papel blanco.[34]

Almacén de datos

Un almacén de datos es un tipo especializado de base de datos relacional, que está optimizado para el análisis y la generación de informes de grandes cantidades de datos. Se puede usar para combinar datos transaccionales de fuentes dispares (como el comportamiento del usuario en una aplicación web, datos de su sistema financiero y de facturación, o gestión de relaciones con los clientes o CRM) para que estén disponibles para el análisis y la toma de decisiones.

Tradicionalmente, configurar, ejecutar y escalar un almacén de datos ha sido complicado y costoso. En AWS, puede aprovechar Amazon Redshift, un servicio de almacenamiento de datos administrado que está diseñado para operar a menos de una décima parte del costo de las soluciones tradicionales.

Escalabilidad

Amazon Redshift logra un almacenamiento eficiente y un rendimiento óptimo de las consultas a través de una combinación de procesamiento paralelo masivo (MPP), almacenamiento de datos en columnas y esquemas de codificación de compresión de datos específicos. Es particularmente adecuado para cargas de trabajo analíticas y de generación de informes en conjuntos de datos muy grandes. La arquitectura MPP de Amazon Redshift le permite aumentar el rendimiento aumentando la cantidad de nodos en su clúster de almacenamiento de datos. Amazon Redshift Spectrum permite consultas SQL de Amazon Redshift contra exabytes de datos en Amazon S3, lo que amplía las capacidades analíticas de Amazon Redshift más allá de los datos almacenados en discos locales en el almacén de datos a datos no estructurados, sin necesidad de cargar o transformar datos.

Alta disponibilidad

Amazon Redshift tiene varias características que mejoran la confiabilidad de su clúster de almacenamiento de datos. Le recomendamos que implemente cargas de trabajo de producción en clústeres de varios nodos, de modo que los datos que se escriben en un nodo se repliquen automáticamente en otros nodos dentro del clúster. Los datos también se respaldan continuamente en Amazon S3. Amazon Redshift monitorea continuamente el estado del clúster y vuelve a replicar automáticamente los datos de las unidades fallidas y reemplaza los nodos según sea necesario. Para obtener más información, consulte elPreguntas frecuentes sobre Amazon Redshift .[35]

Archivado

Antipatrones

Dado que Amazon Redshift es un sistema de administración de bases de datos relacionales (RDBMS) basado en SQL, es compatible con otras aplicaciones RDBMS y herramientas de inteligencia comercial. Aunque Amazon Redshift proporciona la funcionalidad de un RDBMS típico, incluidas las funciones de procesamiento de transacciones en línea (OLTP), no está diseñado para estas cargas de trabajo. Si espera una carga de trabajo de alta simultaneidad que generalmente implica leer y escribir todas las columnas para una pequeña cantidad de registros a la vez, debería considerar usar Amazon RDS o Amazon DynamoDB.

Buscar

La búsqueda a menudo se confunde con la consulta. Una consulta es una consulta de base de datos formal, que se dirige en términos formales a un conjunto de datos específico. La búsqueda permite consultar conjuntos de datos que no están estructurados con precisión. Por esta razón, las aplicaciones que requieren una funcionalidad de búsqueda sofisticada normalmente superarán las capacidades de las bases de datos relacionales o NoSQL. Un servicio de búsqueda se puede utilizar para indexar y buscar en formato de texto libre y estructurado y puede admitir funcionalidades que no están disponibles en otras bases de datos, como clasificación de resultados personalizable, facetas para filtrado, sinónimos, lematización.

En AWS, puede elegir entre Amazon CloudSearch y Amazon Elasticsearch Service (Amazon ES). Amazon CloudSearch es un servicio administrado que requiere poca configuración y se escalará automáticamente. Amazon ES ofrece una API de código abierto y le brinda más control sobre los detalles de configuración. Amazon ES también ha evolucionado para convertirse en algo más que una solución de búsqueda. A menudo se usa como un motor de análisis para casos de uso como análisis de registros, monitoreo de aplicaciones en tiempo real y análisis de flujo de clics.

Archivado

Escalabilidad

Tanto Amazon CloudSearch como Amazon ES utilizan la partición y la replicación de datos para escalar horizontalmente. La diferencia es que con Amazon CloudSearch, no necesita preocuparse por cuántas particiones y réplicas necesita porque el servicio las maneja automáticamente.

Alta disponibilidad

Tanto Amazon CloudSearch como Amazon ES incluyen funciones que almacenan datos de forma redundante en las zonas de disponibilidad. Para más detalles, consulte elBúsqueda en la nube de Amazon [36]y Amazon ES documentación.[37]

Bases de datos de grafos

Una base de datos de gráficos utiliza estructuras de gráficos para consultas. Un gráfico se define como formado por bordes (relaciones), que se relacionan directamente con los nodos (entidades de datos) en el almacén. Las relaciones permiten que los datos del almacén se vinculen directamente, lo que permite la recuperación rápida de estructuras jerárquicas complejas en sistemas relacionales. Por esta razón, las bases de datos de gráficos se construyen específicamente para almacenar y navegar por las relaciones y, por lo general, se usan en casos de uso como redes sociales, motores de recomendación y detección de fraudes, en los que debe poder crear relaciones entre los datos y consultar rápidamente estas relaciones.

Amazon Neptune es un servicio de base de datos de gráficos completamente administrado. Para obtener más información, consulte elPreguntas frecuentes sobre Amazon Neptune.

Escalabilidad

Amazon Neptune es una base de datos de gráficos de alto rendimiento especialmente diseñada y optimizada para procesar consultas de gráficos.

Alta disponibilidad

Amazon Neptune tiene una alta disponibilidad, con réplicas de lectura, recuperación en un momento dado, respaldo continuo en Amazon S3 y replicación en zonas de disponibilidad. Neptune es seguro, con soporte para cifrado en reposo y en tránsito.[38]

Archivado

Gestión de volúmenes crecientes de datos

Las herramientas tradicionales de análisis y almacenamiento de datos ya no pueden proporcionar la agilidad y la flexibilidad necesarias para brindar información comercial relevante. Es por eso que muchas organizaciones están cambiando a una arquitectura de lago de datos. Un lago de datos es un enfoque arquitectónico que le permite almacenar cantidades masivas de datos en una ubicación central para que estén fácilmente disponibles para ser categorizados, procesados, analizados y consumidos por diversos grupos dentro de su organización. Dado que los datos se pueden almacenar tal cual, no tiene que convertirlos a un esquema predefinido y ya no necesita saber qué preguntas hacer sobre sus datos de antemano. Esto le permite seleccionar la tecnología correcta para cumplir con sus requisitos analíticos específicos. Para obtener más información, consulte elCreación de un lago de datos con Amazon Web Services papel blanco.[39]

Eliminación de puntos únicos de falla

Los sistemas de producción normalmente vienen con objetivos definidos o implícitos para el tiempo de actividad. Un sistema tiene alta disponibilidad cuando puede resistir la falla de un componente individual o de varios componentes, como discos duros, servidores y enlaces de red. Para ayudarlo a crear un sistema con alta disponibilidad, puede pensar en formas de automatizar la recuperación y reducir las interrupciones en cada capa de su arquitectura. Para obtener más información acerca de los patrones de diseño de alta disponibilidad, consulte Creación de aplicaciones tolerantes a fallas papel blanco.[40]

Introducción a la redundancia

Los puntos únicos de falla se pueden eliminar mediante la introducción de redundancia, lo que significa que tiene múltiples recursos para la misma tarea. La redundancia se puede implementar en modo de espera o activo.

En *redundancia en espera*, cuando falla un recurso, la funcionalidad se recupera en un recurso secundario con el proceso de conmutación por error. La conmutación por error generalmente requiere algún tiempo antes de que se complete y, durante este período, el recurso permanece no disponible. El recurso secundario puede iniciarse automáticamente solo cuando sea necesario (para reducir costos) o puede estar ya en funcionamiento inactivo (para acelerar la conmutación por error y minimizar las interrupciones). La redundancia en espera se usa a menudo para componentes con estado, como bases de datos relacionales.

Archivado

En *redundancia activa*, las solicitudes se distribuyen a varios recursos informáticos redundantes. Cuando uno de ellos falla, el resto simplemente puede absorber una mayor parte de la carga de trabajo. En comparación con la redundancia en espera, la redundancia activa puede lograr un mejor uso y afectar a una población más pequeña cuando hay una falla.

Detectar falla

Debe apuntar a construir la mayor automatización posible tanto en la detección como en la reacción ante fallas. Puede utilizar servicios como ELB y Amazon Route 53 para configurar comprobaciones de estado y enmascarar errores mediante el enrutamiento del tráfico a puntos finales en buen estado. Además, puede reemplazar los nodos en mal estado automáticamente usando Auto Scaling o usando el Función de recuperación automática de Amazon EC2 o servicios como AWS Elastic Beanstalk.[41] No será posible predecir todos los posibles escenarios de falla el primer día. Asegúrese de recopilar suficientes registros y métricas para comprender el comportamiento normal del sistema. Una vez que comprenda eso, podrá configurar alarmas para intervención manual o respuesta automática.

Diseñar buenos controles de salud

La configuración de las comprobaciones de estado adecuadas para su aplicación ayuda a determinar su capacidad para responder correcta y rápidamente a una variedad de escenarios de falla. Especificar una verificación de estado incorrecta puede reducir la disponibilidad de su aplicación.

En una aplicación típica de tres niveles, configura comprobaciones de estado en ELB. Diseñe sus comprobaciones de estado con el objetivo de evaluar de forma fiable el estado de los nodos de back-end. Una simple comprobación de estado de TCP no detectará si la instancia en sí está en buen estado, pero el proceso del servidor web se ha bloqueado. En su lugar, debe evaluar si el servidor web puede devolver una respuesta HTTP 200 para una solicitud simple.

En la capa, puede que no sea una buena idea configurar un *control de salud profundo*, que es una prueba que depende de otras capas de su aplicación para tener éxito, ya que pueden generarse falsos positivos. Por ejemplo, si su verificación de estado también evalúa si la instancia puede conectarse a una base de datos de back-end, corre el riesgo de marcar todos sus servidores web como en mal estado cuando ese nodo de la base de datos deja de estar disponible en breve. Un enfoque en capas suele ser el mejor. Una verificación de estado profunda podría ser adecuada para implementar en el nivel de Amazon Route 53. Al ejecutar una verificación más holística que determina si ese entorno puede proporcionar realmente la funcionalidad requerida, puede configurar Amazon Route 53 para que realice una conmutación por error a una versión estática de su sitio web hasta que su base de datos esté en funcionamiento nuevamente.

Archivado

Almacenamiento de datos duradero

Su aplicación y sus usuarios crearán y mantendrán una variedad de datos. Es fundamental que su arquitectura proteja tanto la disponibilidad como la integridad de los datos. La replicación de datos es la técnica que introduce copias redundantes de datos. Puede ayudar a escalar horizontalmente la capacidad de lectura, pero también aumenta la durabilidad y disponibilidad de los datos. La replicación puede ocurrir en algunos modos diferentes.

*replicación síncrona*solo reconoce una transacción después de que se haya almacenado de forma duradera tanto en la ubicación principal como en sus réplicas. Es ideal para proteger la integridad de los datos en caso de falla del nodo principal. La replicación síncrona también puede escalar la capacidad de lectura para consultas que requieren los datos más actualizados (coherencia fuerte). El inconveniente de la replicación síncrona es que el nodo principal está acoplado a las réplicas. No se puede reconocer una transacción antes de que todas las réplicas hayan realizado la escritura. Esto puede comprometer el rendimiento y la disponibilidad, especialmente en topologías que se ejecutan en conexiones de red poco confiables o de alta latencia. Por la misma razón, no se recomienda mantener muchas réplicas síncronas.

Independientemente de la durabilidad de su solución, esto no reemplaza a las copias de seguridad. La replicación síncrona almacena de manera redundante todas las actualizaciones de sus datos, incluso aquellas que son el resultado de errores de software o errores humanos. Sin embargo, particularmente para los objetos almacenados en Amazon S3, puede usarversionado para conservar, recuperar y restaurar cualquiera de sus versiones.[42]Con el control de versiones, puede recuperarse tanto de las acciones no deseadas del usuario como de las fallas de la aplicación.

*replicación asíncrona*desacopla el nodo principal de sus réplicas a expensas de introducir un retraso en la replicación. Esto significa que los cambios en el nodo principal no se reflejan inmediatamente en sus réplicas. Las réplicas asíncronas se utilizan para escalar horizontalmente la capacidad de lectura del sistema para consultas que pueden tolerar ese retraso de replicación. También se puede usar para aumentar la durabilidad de los datos cuando se puede tolerar cierta pérdida de transacciones recientes durante una conmutación por error. Por ejemplo, puede mantener una réplica asíncrona de una base de datos en una región de AWS independiente como solución de recuperación ante desastres.

*Replicación basada en quórum*combina la replicación síncrona y asíncrona para superar los desafíos de los sistemas de bases de datos distribuidas a gran escala.

La replicación en varios nodos se puede administrar definiendo una cantidad mínima de nodos que deben participar en una operación de escritura exitosa. Una discusión detallada de

Archivado

Los almacenes de datos distribuidos están más allá del alcance de este documento. Para obtener más información sobre los almacenes de datos distribuidos y el conjunto básico de principios para un sistema de base de datos ultraescalable y altamente confiable, consulte elamazon dínamo papel blanco.[43]

Es importante comprender dónde encaja cada tecnología que está utilizando en estos modelos de almacenamiento de datos. Su comportamiento durante varios escenarios de conmutación por error o copia de seguridad/restauración debe alinearse con su objetivo de punto de recuperación (RPO) y su objetivo de tiempo de recuperación (RTO). Debe determinar cuántos datos espera perder y qué tan rápido necesita reanudar las operaciones. Por ejemplo, el motor de Redis para Amazon ElastiCache admite la replicación con conmutación por error automática, pero la replicación del motor de Redis es asíncrona. Durante una conmutación por error, es muy probable que se pierdan algunas transacciones recientes. Sin embargo, Amazon RDS, con su función Multi-AZ, está diseñado para proporcionar replicación síncrona para mantener los datos en el nodo en espera actualizados con el principal.

Resiliencia automatizada de varios centros de datos

Las aplicaciones críticas para el negocio también necesitan protección contra escenarios de interrupción que afectan a más de un solo disco, servidor o bastidor. En una infraestructura tradicional, normalmente tiene un plan de recuperación ante desastres para permitir la conmutación por error a un segundo centro de datos distante en caso de una interrupción importante en el principal. Debido a la gran distancia entre los dos centros de datos, la latencia hace que no sea práctico mantener copias sincrónicas de los datos entre centros de datos. Como resultado, una conmutación por error seguramente provocará la pérdida de datos o un proceso de recuperación de datos muy costoso. Esto hace que la conmutación por error sea un procedimiento arriesgado y no siempre suficientemente probado. Sin embargo, este es un modelo que brinda una eficiente protección contra un riesgo de baja probabilidad pero de gran impacto, como una catástrofe natural que derriba toda su infraestructura durante mucho tiempo. Recuperación ante desastres de AWS papel blanco.[44]

Una interrupción más corta en un centro de datos es un escenario más probable. Para interrupciones cortas en las que no se prevé que la duración de la falla sea larga, la opción de realizar una conmutación por error es difícil y generalmente se evita. En AWS, es posible adoptar una protección más simple y eficiente contra este tipo de fallas. Cada región de AWS contiene varias ubicaciones distintas, o*Zonas de disponibilidad*. Cada zona de disponibilidad está diseñada para ser independiente de las fallas en otras zonas de disponibilidad. Una zona de disponibilidad es un centro de datos y, en algunos casos, una zona de disponibilidad consta de varios centros de datos. Las zonas de disponibilidad dentro de una región proporcionan una red económica y de baja latencia.

Archivado

conectividad con otras zonas de la misma Región. Esto le permite replicar sus datos en los centros de datos de manera síncrona para que la conmutación por error se pueda automatizar y sea transparente para sus usuarios.

También es posible implementar redundancia activa. Por ejemplo, una flota de servidores de aplicaciones puede distribuirse en varias zonas de disponibilidad y adjuntarse a ELB. Cuando las instancias EC2 de una zona de disponibilidad en particular fallan en sus controles de estado, ELB deja de enviar tráfico a esos nodos. Además, AWS Auto Scaling garantiza que la cantidad correcta de instancias EC2 esté disponible para ejecutar su aplicación, lanzando y terminando instancias en función de la demanda y definidas por sus políticas de escalado. Si su aplicación no requiere una degradación del rendimiento a corto plazo debido a una falla en la zona de disponibilidad, su arquitectura debe ser estáticamente estable, lo que significa que no requiere un cambio en el comportamiento de su carga de trabajo para tolerar fallas. En este escenario,[45]

Muchos de los servicios de nivel superior en AWS están inherentemente diseñados de acuerdo con el principio de múltiples zonas de disponibilidad (Multi-AZ). Por ejemplo, Amazon RDS brinda alta disponibilidad y soporte de conmutación por error automático para instancias de base de datos que utilizan implementaciones Multi-AZ, mientras que con Amazon S3 y Amazon DynamoDB sus datos se almacenan de forma redundante en varias instalaciones.

Aislamiento de fallas y escalado horizontal tradicional

Aunque el patrón de redundancia activa es excelente para equilibrar el tráfico y manejar las interrupciones de instancias o zonas de disponibilidad, no es suficiente si hay algo dañino en las solicitudes mismas. Por ejemplo, podría haber escenarios en los que todas las instancias se vean afectadas. Si una solicitud en particular desencadena un error que hace que el sistema falle, entonces la persona que llama puede desencadenar una falla en cascada al intentar repetidamente la misma solicitud en todas las instancias.

Fragmentación aleatoria

Una mejora de aislamiento de fallas que puede realizar en el escalado horizontal tradicional se llama *fragmentación*. De manera similar a la técnica utilizada tradicionalmente con los sistemas de almacenamiento de datos, en lugar de distribuir el tráfico de todos los clientes en todos los nodos, puede agrupar las instancias en fragmentos. Por ejemplo, si tiene ocho instancias para su servicio, puede crear cuatro fragmentos de dos instancias cada uno (dos instancias para alguna redundancia dentro de cada fragmento) y distribuir cada cliente a un fragmento específico. De esta manera, usted

Archivado

son capaces de reducir el impacto en los clientes en proporción directa a la cantidad de fragmentos que tiene. Sin embargo, algunos clientes aún se verán afectados, por lo que la clave es hacer que el cliente sea tolerante a fallas. Si el cliente puede probar todos los puntos finales en un conjunto de recursos fragmentados hasta que uno tenga éxito, obtendrá una mejora espectacular. Esta técnica se conoce como *fragmentación aleatoria*. Para más información sobre esta técnica ver el Shuffle Sharding: Aislamiento de fallas masivo y mágico entrada en el blog.[46]

Archivado

Optimizar por costo

Cuando traslada sus arquitecturas existentes a la nube, puede reducir los gastos de capital y generar ahorros como resultado de las economías de escala de AWS. Al iterar y usar más capacidades de AWS, puede aprovechar más oportunidades para crear arquitecturas en la nube optimizadas en función de los costos. Para obtener más información acerca de cómo optimizar los costos con la computación en la nube de AWS, consulte laOptimización de costos con AWS papel blanco.[47]

Tamaño correcto

AWS ofrece una amplia gama de tipos de recursos y configuraciones para muchos casos de uso. Por ejemplo, servicios como Amazon EC2, Amazon RDS, Amazon Redshift y Amazon ES ofrecen muchos tipos de instancias. En algunos casos, debe seleccionar el tipo más económico que se adapte a los requisitos de su carga de trabajo. En otros casos, el uso de menos instancias de un tipo de instancia más grande puede generar un costo total más bajo o un mejor rendimiento. Debe comparar el entorno de su aplicación y seleccionar el tipo de instancia adecuado en función de cómo su carga de trabajo utilice la CPU, la RAM, la red, el tamaño del almacenamiento y las operaciones de E/S.

Del mismo modo, puede reducir los costos seleccionando la solución de almacenamiento adecuada para sus necesidades. Por ejemplo, Amazon S3 ofrece una variedad de clases de almacenamiento, que incluyen Estándar, Redundancia reducida y Acceso estándar o poco frecuente. Otros servicios, como Amazon EC2, Amazon RDS y Amazon ES, admiten diferentes tipos de volumen de EBS (SSD magnético, de uso general, SSD IOPS aprovisionado) que debe evaluar.

Con el tiempo, puede continuar reduciendo costos con monitoreo y etiquetado continuos. Al igual que el desarrollo de aplicaciones, la optimización de costos es un proceso iterativo. Debido a que su aplicación y su uso evolucionarán con el tiempo, y debido a que AWS itera con frecuencia y lanza regularmente nuevas opciones, es importante evaluar continuamente su solución.

AWS proporciona herramientas para ayudarlo a identificar esas oportunidades de ahorro de costos y mantener sus recursos en el tamaño adecuado.[48]Para que los resultados de esas herramientas sean fáciles de interpretar, debe definir e implementar una política de etiquetado para sus recursos de AWS. Puede hacer que el etiquetado forme parte de su proceso de compilación y automatizarlo con las herramientas de administración de AWS, como AWS Elastic Beanstalk y AWS OpsWorks. También puede utilizar las reglas administradas proporcionadas por AWS Config para evaluar si se aplican etiquetas específicas a sus recursos o no.

Archivado

Elasticidad

Otra forma de ahorrar dinero con AWS es aprovechar la elasticidad de la plataforma. Planifique implementar Auto Scaling para tantas cargas de trabajo de Amazon EC2 como sea posible, de modo que pueda escalar horizontalmente cuando sea necesario y reducir y reducir automáticamente sus gastos cuando ya no necesite esa capacidad. Además, puedesautomatizar la desactivación de cargas de trabajo que no son de producción cuando no está en uso.[49] En última instancia, considere qué cargas de trabajo informáticas podría implementar en AWS Lambda para que nunca pague por recursos inactivos o redundantes.

Siempre que sea posible, reemplace las cargas de trabajo de Amazon EC2 con servicios administrados de AWS que no requieran que tome decisiones de capacidad (como ELB, Amazon CloudFront, Amazon SQS, Amazon Kinesis Firehose, AWS Lambda, Amazon SES, Amazon CloudSearch o Amazon EFS).) o permitirle modificar fácilmente la capacidad cuando sea necesario (como Amazon DynamoDB, Amazon RDS o Amazon ES).

Aproveche la variedad de opciones de compra

Los precios de las instancias bajo demanda de Amazon EC2 le brindan la máxima flexibilidad sin compromisos a largo plazo. Otras dos instancias EC2 que pueden ayudarlo a reducir el gasto son *Instancias reservadas* y *Instancias puntuales*.

Instancias reservadas

Las instancias reservadas de Amazon EC2 le permiten reservar la capacidad informática de Amazon EC2 a cambio de una tarifa por hora con un descuento significativo en comparación con los precios de las instancias bajo demanda. Esto es ideal para aplicaciones con requisitos de capacidad mínima predecibles. Puede aprovechar herramientas como AWS Trusted Advisor o los informes de uso de Amazon EC2 para identificar los recursos informáticos que utiliza con más frecuencia y que debería considerar reservar. Según sus compras de Instancia reservada, los descuentos se reflejarán en la factura mensual. Técnicamente, no hay diferencia entre una instancia EC2 bajo demanda y una instancia reservada. La diferencia radica en la forma en que paga por las instancias que reserva.

También existen opciones de capacidad reservada para otros servicios (p. ej., Amazon Redshift, Amazon RDS, Amazon DynamoDB y Amazon CloudFront).

> **Consejo**: no debe comprometerse con las compras de instancias reservadas antes de haber evaluado suficientemente su aplicación en producción. Después de usted

Archivado

ha adquirido capacidad reservada, puede utilizar los informes de utilización de instancias reservadas para asegurarse de que sigue aprovechando al máximo su capacidad reservada.

Instancias de spot

Para cargas de trabajo menos estables, considere usar instancias de spot. Las instancias de spot de Amazon EC2 le permiten utilizar la capacidad informática sobrante de Amazon EC2. Dado que las instancias puntuales suelen estar disponibles con un descuento en comparación con los precios bajo demanda, puede reducir significativamente el costo de ejecución de sus aplicaciones.

Las instancias puntuales le permiten solicitar instancias EC2 no utilizadas, lo que puede reducir significativamente los costos de Amazon EC2. El precio por hora de una instancia de spot (de cada tipo de instancia en cada zona de disponibilidad) lo establece Amazon EC2 y se ajusta gradualmente en función de la oferta y la demanda a largo plazo de las instancias de spot. Su instancia de Spot se ejecuta siempre que haya capacidad disponible y el precio máximo por hora para su solicitud supere el precio de Spot.

Como resultado, las instancias de spot son ideales para cargas de trabajo que pueden tolerar interrupciones. Sin embargo, también puede utilizar instancias de spot cuando necesite una disponibilidad más predecible. Por ejemplo, puede combinar instancias reservadas, bajo demanda y puntuales para combinar una capacidad mínima predecible con *oportunista* acceso a recursos informáticos adicionales, según el precio de mercado al contado. Esta es una forma excelente y rentable de mejorar el rendimiento o el rendimiento de la aplicación.

Archivado

almacenamiento en caché

El almacenamiento en caché es una técnica que almacena datos previamente calculados para uso futuro. Esta técnica se utiliza para mejorar el rendimiento de la aplicación y aumentar la rentabilidad de una implementación. Se puede aplicar en múltiples capas de una arquitectura de TI.

Almacenamiento en caché de datos de aplicaciones

Las aplicaciones se pueden diseñar para que almacenen y recuperen información de cachés en memoria rápidos y administrados. La información almacenada en caché puede incluir los resultados de consultas de base de datos intensivas de E/S o el resultado de un procesamiento computacionalmente intensivo. Cuando el conjunto de resultados no se encuentra en la memoria caché, la aplicación puede calcularlo o recuperarlo de una base de datos o contenido de terceros costoso y que muta lentamente, y almacenarlo en la memoria caché para solicitudes posteriores. Sin embargo, cuando se encuentra un conjunto de resultados en la caché, la aplicación puede usar ese resultado directamente, lo que mejora la latencia para los usuarios finales y reduce la carga en los sistemas de back-end. Su aplicación puede controlar cuánto tiempo permanece válido cada elemento almacenado en caché. En algunos casos, incluso unos segundos de almacenamiento en caché para objetos muy populares pueden ayudar en una disminución drástica de la carga de su base de datos.

Amazon ElastiCache es un servicio web que facilita la implementación, el funcionamiento y el escalado de una caché en memoria en la nube. Admite dos motores de almacenamiento en memoria caché de código abierto: Memcached y Redis. Para obtener más detalles sobre cómo seleccionar el motor adecuado para su carga de trabajo, así como una descripción de los patrones de diseño comunes de ElastiCache, consulte la[Rendimiento a escala con Amazon ElastiCache](#) papel blanco.[50]

Amazon DynamoDB Accelerator (DAX) es una caché en memoria totalmente administrada y de alta disponibilidad para DynamoDB que ofrece mejoras de rendimiento de milisegundos a microsegundos, para un alto rendimiento. DAX agrega aceleración en memoria a sus tablas de DynamoDB sin necesidad de administrar la invalidación de caché, el llenado de datos o la administración de clústeres.

Almacenamiento en caché perimetral

Las copias de contenido estático (imágenes, archivos CSS o transmisión de video pregrabado) y contenido dinámico (HTML receptivo, video en vivo) se pueden almacenar en caché en una ubicación de borde de Amazon CloudFront, que es una CDN con múltiples puntos de presencia en todo el mundo. El almacenamiento en caché perimetral permite que el contenido sea servido por una infraestructura que está más cerca de

Archivado

espectadores, lo que reduce la latencia y le brinda las tasas de transferencia de datos altas y sostenidas necesarias para entregar grandes objetos populares a los usuarios finales a escala.

Las solicitudes de su contenido se enrutan de forma inteligente a Amazon S3 o a sus servidores de origen. Si el origen se ejecuta en AWS, las solicitudes se transfieren a través de rutas de red optimizadas para una experiencia más confiable y consistente. Puede usar Amazon CloudFront para entregar todo su sitio web, incluido el contenido que no se puede almacenar en caché. En este caso, el beneficio es que Amazon CloudFront reutiliza las conexiones existentes entre el borde de Amazon CloudFront y el servidor de origen, lo que reduce la latencia de configuración de la conexión para cada solicitud de origen. También se aplican otras optimizaciones de conexión para evitar cuellos de botella en Internet y aprovechar al máximo el ancho de banda disponible entre la ubicación de borde y el espectador. Esto significa que Amazon CloudFront puede agilizar la entrega de su contenido dinámico y brindar a sus espectadores un servicio consistente y confiable, pero personalizado. experiencia al navegar por su aplicación web. Amazon CloudFront también aplica los mismos beneficios de rendimiento a las solicitudes de descarga que los aplicados a las solicitudes de descarga de contenido dinámico.

Seguridad

La mayoría de las herramientas y técnicas de seguridad con las que quizás ya esté familiarizado en una infraestructura de TI tradicional se pueden usar en la nube. Al mismo tiempo, AWS le permite mejorar su seguridad de varias formas. AWS es una plataforma que permite formalizar el diseño de controles de seguridad en la propia plataforma. Simplifica el uso del sistema para los administradores y su departamento de TI, y hace que su entorno sea mucho más fácil de auditar de manera continua. Para obtener una visión detallada de cómo puede lograr un alto nivel de control de la seguridad, consulte el Seguridad a escala: Gobernanza en AWS [51] y el Prácticas recomendadas de seguridad de AWS libros blancos.

Utilice las funciones de AWS para la defensa en profundidad

AWS proporciona muchas funciones que pueden ayudarlo a crear arquitecturas que cuentan con métodos de defensa en profundidad. A partir del nivel de red, puede crear una topología de VPC que aísle partes de la infraestructura mediante el uso de subredes, grupos de seguridad y controles de enrutamiento. Los servicios como AWS WAF, un firewall de aplicaciones web, pueden ayudarlo a proteger sus aplicaciones web de la inyección SQL y otras vulnerabilidades en el código de su aplicación. Para el control de acceso, puede utilizar IAM para definir un conjunto granular de políticas y asignarlas a usuarios, grupos y recursos de AWS. Finalmente, la nube de AWS ofrece muchas opciones para proteger sus datos, ya sea que estén en tránsito o en reposo.con

Archivado

cifrado .[53]Para obtener más información sobre todas las características de seguridad de AWS disponibles, consulte la Seguridad en la nube de AWS página en el sitio web de AWS.[54]

Comparta la responsabilidad de seguridad con AWS

AWS opera bajo un modelo de responsabilidad de seguridad compartida: AWS es responsable de la seguridad de la infraestructura de la nube subyacente y usted es responsable de proteger las cargas de trabajo que implementa en AWS. Esto lo ayuda a reducir el alcance de su responsabilidad y concentrarse en sus competencias principales mediante el uso de los servicios administrados de AWS. Por ejemplo, cuando utiliza servicios como Amazon RDS y Amazon ElastiCache, los parches de seguridad se aplican automáticamente a sus ajustes de configuración. Esto no solo reduce los gastos generales operativos de su equipo, sino que también podría reducir su exposición a las vulnerabilidades.

Reducir el acceso privilegiado

Cuando sus servidores son recursos programables, obtiene muchos beneficios de seguridad. La capacidad de cambiar sus servidores cuando lo necesite le permite eliminar la necesidad de acceso del sistema operativo invitado a los entornos de producción. Si una instancia experimenta un problema, puede cancelarla y reemplazarla de forma automática o manual. Sin embargo, antes de reemplazar las instancias, debe recopilar y almacenar de forma centralizada los datos de registro que pueden ayudarlo a recrear problemas en su entorno de desarrollo e implementarlos como soluciones a través de su proceso de implementación continua. Este enfoque garantiza que los datos de registro ayuden con la resolución de problemas y aumenten la conciencia sobre los eventos de seguridad. Esto es particularmente importante en un entorno informático elástico donde los servidores son temporales. Puede utilizar Amazon CloudWatch Logs para recopilar esta información. Donde no tienes acceso directo,Administrador de sistemas de AWS[55]para tener una vista unificada y automatizar acciones en grupos de recursos. Puede integrar estas solicitudes con su sistema de emisión de boletos, de modo que las solicitudes de acceso sean rastreadas y manejadas dinámicamente solo después de su aprobación.

Otro riesgo de seguridad común es el uso de credenciales o cuentas de servicio almacenadas a largo plazo. En un entorno tradicional, a las cuentas de servicio a menudo se les asignan credenciales a largo plazo que se almacenan en un archivo de configuración. En AWS, puede usar roles de IAM para otorgar permisos a aplicaciones que se ejecutan en instancias EC2 mediante el uso de credenciales a corto plazo, que se distribuyen y rotan automáticamente. Para las aplicaciones móviles, puede usar Amazon Cognito para permitir que los dispositivos de los clientes accedan a los recursos de AWS a través de tokens temporales con permisos detallados. como una AWS

Archivado

Usuario de Management Console, puede proporcionar acceso federado de manera similar a través de tokens temporales en lugar de crear usuarios de IAM en su cuenta de AWS. Luego, cuando un empleado deja su organización y se elimina del directorio de identidad de su organización, ese empleado también pierde automáticamente el acceso a sus cuentas de AWS.

Seguridad como código

Los marcos de seguridad, las reglamentaciones y las políticas organizacionales tradicionales definen los requisitos de seguridad relacionados con elementos tales como reglas de firewall, controles de acceso a la red, subredes internas/externas y fortalecimiento del sistema operativo. También puede implementarlos en un entorno de AWS, pero ahora tiene la oportunidad de capturarlos todos en una plantilla que define un*Ambiente Dorado*. AWS CloudFormation utiliza esta plantilla e implementa sus recursos de acuerdo con su política de seguridad. Puede reutilizar las mejores prácticas de seguridad entre varios proyectos, como parte de su proceso de integración continua. Puede realizar pruebas de seguridad como parte de su ciclo de lanzamiento y descubrir automáticamente brechas en las aplicaciones y desviarse de su política de seguridad.

Además, para mayor control y seguridad, las plantillas de AWS CloudFormation se pueden importar como*productos*en[Catálogo de servicios de AWS](#).[56]Esto le permite administrar de forma centralizada sus recursos para respaldar requisitos de cumplimiento, seguridad y gobernanza consistentes, al tiempo que permite a sus usuarios implementar rápidamente solo los servicios de TI aprobados que necesitan. Aplica permisos de IAM para controlar quién puede ver y modificar sus productos, y define restricciones para restringir las formas en que se pueden implementar recursos específicos de AWS para un producto.

Auditoría en tiempo real

Probar y auditar su entorno es clave para moverse rápido mientras se mantiene seguro. Los enfoques tradicionales que implican verificaciones periódicas (y a menudo manuales o basadas en muestras) no son suficientes, especialmente en entornos ágiles donde el cambio es constante. En AWS, puede implementar el monitoreo continuo y la automatización de los controles para minimizar la exposición a los riesgos de seguridad. Los servicios como AWS Config, Amazon Inspector y AWS Trusted Advisor monitorean continuamente el cumplimiento o las vulnerabilidades, lo que le brinda una visión general clara de qué recursos de TI cumplen y cuáles no. Con las reglas de AWS Config, también sabe si un recurso no cumplió, incluso durante un breve período de tiempo, lo que hace que las auditorías puntuales y periódicas sean muy efectivas. Tú

Archivado

poderimplementar un registro extenso para sus aplicaciones (mediante Amazon CloudWatch Logs) y para las llamadas a la API de AWS reales al habilitar AWS CloudTrail.[57]

AWS CloudTrail es un servicio web que registra llamadas API a servicios de AWS compatibles en su cuenta de AWS y entrega un archivo de registro a su depósito S3. Los datos de registro pueden almacenarse de manera inmutable y procesarse automáticamente para enviar una notificación o tomar una acción en su nombre, protegiendo a su organización del incumplimiento. Puede usar AWS Lambda, Amazon EMR, Amazon ES, Amazon Athena o herramientas de terceros de AWS Marketplace para escanear datos de registro y detectar eventos como permisos no utilizados, uso excesivo de cuentas con privilegios, uso de claves, inicios de sesión anómalos, infracciones de políticas y errores del sistema. abuso.

Archivado

Conclusión

Cuando diseña su arquitectura en la nube de AWS, es importante tener en cuenta los importantes principios y patrones de diseño disponibles en AWS, incluido cómo seleccionar la base de datos adecuada para su aplicación y cómo diseñar aplicaciones que puedan escalar horizontalmente y con alta disponibilidad. Debido a que cada implementación es única, debe evaluar cómo aplicar esta guía a su implementación. El tema de las arquitecturas de computación en la nube es amplio y está en continua evolución. Puede mantenerse actualizado con los últimos cambios y adiciones a las ofertas de la nube de AWS con el material disponible en elSitio web de AWS [58]y elOfertas de capacitación y certificación de AWS .[59]

Colaboradores

Estas personas contribuyeron a este documento:

- Andreas Chatzakis, Gerente, Arquitectura de soluciones de AWS
- Paul Armstrong, arquitecto principal de soluciones, AWS

Otras lecturas

Para ver más ejemplos de arquitectura, consulte elCentro de arquitectura de AWS .[60]

Para las aplicaciones que ya se ejecutan en AWS, le recomendamos que revise elMarco de buena arquitectura de AWS documento técnico, que proporciona un modelo de evaluación estructurado.[61]

Para obtener información que lo ayude a validar su preparación operativa, consulte la lista de verificación operativa de AWS .[62]

Archivado

Revisiones de documentos

Fecha	Descripción
octubre 2018	Revisión y actualización de documentos
Junio del 2013	Primera publicación

notas

1 http://d0.awsstatic.com/whitepapers/architecture/AWS_Well-Architected_Framework.pdf

2 https://aws.amazon.com/about-aws/

3 https://aws.amazon.com/about-aws/global-infrastructure

4 Por ejemplo, consulte el controlador de sesión PHP Amazon DynamoDB (http://docs.aws.amazon.com/aws-sdk-php/v3/guide/service/dynamodb-sessionhandler.html) y el controlador de sesión Tomcat Amazon DynamoDB (http://docs.aws.amazon.com/AWSSdkDocsJava/latest//DeveloperGuide/java-dgtomcat-session-manager.html)

5 https://docs.aws.amazon.com/elasticloadbalancing/latest/application/load-balancer-target-groups.html#sticky-sessions

6 https://d0.awsstatic.com/whitepapers/Big_Data_Analytics_Options_on_AWS.pdf

7 https://d1.awsstatic.com/whitepapers/core-tenets-of-iot1.pdf

8 http://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-instance-metadatos.html

9 http://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/template-recursos-personalizados-lambda.html

10 http://docs.aws.amazon.com/AWSEC2/latest/UserGuide/AMIs.html

11 http://docs.aws.amazon.com/elasticbeanstalk/latest/dg/concepts.platforms.html

Archivado

12 http://docs.aws.amazon.com/elasticbeanstalk/latest/dg/ebextensions.html

13 https://d0.awsstatic.com/whitepapers/overview-of-deployment-options-on-aws.pdf

14 https://d0.awsstatic.com/whitepapers/managing-your-aws-infrastructure-at-escala.pdf

15 https://d0.awsstatic.com/whitepapers/DevOps/infrastructure-as-code.pdf

dieciséis https://docs.aws.amazon.com/lambda/latest/dg/automating-deployment.html

17 https://aws.amazon.com/elasticbeanstalk/

18 http://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-instance-recover.html

19 https://aws.amazon.com/ec2/systems-manager/

20 https://aws.amazon.com/autoescalado/

21 http://docs.aws.amazon.com/AmazonCloudWatch/latest/DeveloperGuide/AlarmThatSendsEmail.html

22 http://docs.aws.amazon.com/AmazonCloudWatch/latest/DeveloperGuide/WhatIsCloudWatchEvents.html

http://docs.aws.amazon.com/lambda/latest/dg/with-scheduled-events.html

2 https://aws.amazon.com/answers/security/aws-waf-security-automations/

25 https://www.awsarchitectureblog.com/2015/03/backoff.html

26 http://aws.amazon.com/products/

28 https://d0.awsstatic.com/whitepapers/AWS_Serverless_Multi-Tier_Architectures.pdf

29 http://docs.aws.amazon.com/lambda/latest/dg/lambda-edge.html

30 http://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/CHAP_MejoresPrácticas.html

Archivado

31 https://aws.amazon.com/nosql/

32 https://aws.amazon.com/dynamodb/dax/

33 http://docs.aws.amazon.com/amazondynamodb/latest/developerguide/BestPractices.html

34 https://d0.awsstatic.com/whitepapers/migration-best-practices-rdbms-to-dynamodb.pdf

35 https://aws.amazon.com/redshift/faqs/

36 https://aws.amazon.com/documentation/cloudsearch/

37 https://aws.amazon.com/documentation/elasticsearch-service/

38 https://aws.amazon.com/neptuno/

39 https://d0.awsstatic.com/whitepapers/Storage/data-lake-on-aws.pdf

40 https://d0.awsstatic.com/whitepapers/aws-building-fault-tolerant-applications.pdf

41 http://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-instance-recover.html

42 http://docs.aws.amazon.com/amazonS3/latest/dev/Versioning.html

43 http://www.allthingsdistributed.com/2007/10/amazons_dynamo.html

44 http://media.amazonwebservices.com/AWS_Disaster_Recovery.pdf

45 https://aws.amazon.com/architecture/well-architeched/

46 http://www.awsarchitectureblog.com/2014/04/shuffle-sharding.html

47 https://d0.awsstatic.com/whitepapers/Cost_Optimization_with_AWS.pdf

48 http://docs.aws.amazon.com/awsaccountbilling/latest/aboutv2/monitoring-costos.html

49 http://docs.aws.amazon.com/AmazonCloudWatch/latest/DeveloperGuía/Uso de acciones de alarma.html

Archivado

50https://d0.awsstatic.com/whitepapers/performance-at-scale-with-amazon-elasticache.pdf

51https://d0.awsstatic.com/whitepapers/compliance/AWS_Security_a_escala_Gobernanza_en_AWS_Whitepaper.pdf

52https://d0.awsstatic.com/whitepapers/aws-security-best-practices.pdf

53https://d0.awsstatic.com/whitepapers/aws-security-best-practices.pdf

54http://aws.amazon.com/security

55https://aws.amazon.com/systems-manager/

56https://aws.amazon.com/servicecatalog/

57https://d0.awsstatic.com/whitepapers/compliance/AWS_Security_at_Scale_Logging_in_AWS_Whitepaper.pdf

58https://aws.amazon.com/

59https://aws.amazon.com/entrenamiento/

60https://aws.amazon.com/architecture

61http://d0.awsstatic.com/whitepapers/architecture/AWS_Well-Architected_Framework.pdf

https://d0.awsstatic.com/whitepapers/aws-operational-checklists.pdf

Archivado